PhotonicsAI — AI-Driven Science and Autonomous Laboratories

Homework 3: Hardware in the Loop

Name: Erik Mirzoyan

Code: `https://github.com/10mirzoyan71/photonicsai-hw3`

Instrument: one of the lab cameras, a **Hikrobot MV-CS028-10UM** industrial camera (monochrome, USB 3.0, USB3 Vision / GenICam). **Interface:** Hikrobot's MVS SDK (the `MvCameraControl` library, used through the Python interface that MVS installs; MVS 5.1.0, camera SDK 4.8.2.2). The code: `camera/` (the connector), `mcp_server/` (the MCP server), `agent/` (the agent system with the hardware turn), `tests/`.

Project. The agent system turns the lab camera into a characterised measuring instrument on its own, from a plain-language request: it finds working settings (exposure without saturation, free of the room lights' flicker), measures the camera's noise model (conversion gain, read noise, full well, dark level) and its stability, and then puts these measured numbers into the camera model of the Homework 2 focusing bench to predict the focusing enhancement that a wavefront-shaping experiment would reach with this camera. The lab has no laser for this homework, so the scene is a white surface under the room lights; the lab's DMDs with this camera would be the next step towards running the Homework 2 experiment on real hardware.

(a) The connector

`camera/base.py` defines the interface every camera implements: `info()`, `set_exposure(us)`, `set_gain(dB)`, `set_roi(x, y, w, h)`, `grab()` (one frame as a `uint16` array) and `close()`. `camera/hikrobot_camera.py` implements it with the MVS SDK:

connector method	MVS SDK call / camera feature (GenICam name)
open	`MV_CC_EnumDevices` → `MV_CC_CreateHandle` → `MV_CC_OpenDevice` (exclusive)
setup	`ExposureAuto`=Off, `GainAuto`=Off, `ADCBitDepth`=deepest (12 bit), `BlackLevel`=180, `PixelFormat`=Mono12 (else Mono10, Mono8), `TriggerMode`=On, `TriggerSource`=Software, `MV_CC_StartGrabbing`
`set_exposure` / `set_gain`	`ExposureTime` [µs] / `Gain` [dB] (float features, value read back)
`set_roi`	`OffsetX`, `OffsetY`, `Width`, `Height` (camera increments; acquisition stopped)
`grab`	`TriggerSoftware` → `MV_CC_GetImageBuffer` → copy → `MV_CC_FreeImageBuffer`
`info`	`WidthMax`, `HeightMax`, ranges of `ExposureTime` and `Gain`, current settings

With the software trigger every frame is exposed after the command that precedes it, with the settings just applied, so a measurement can never come from a stale frame. Sensor size, bit depth and ranges are read from the camera, not assumed. `camera/sim_camera.py` implements the same interface with a simulated 12-bit camera (shot noise, read noise, dark current, fixed-pattern noise, saturation, a slowly drifting light source) looking at a laser speckle pattern (`sim`) or at a white surface under room lights that flicker at 100 Hz (`sim-room`; a frame integrates the flicker from a random phase, so its mean fluctuates unless the exposure spans whole 10 ms periods). It is used for the tests and for the dry runs below; a photon-transfer measurement on it recovers its conversion gain (2.38 vs. 2.40 e^-/DN) and read noise (5.97 vs. 6.0 e^-), which checks that the analysis the agent uses is sound.

Check of the SDK on the laptop (without the camera): `tests/check_mvs_sdk.py` loads the MVS library and checks every function, structure, field and pixel-format code the connector uses: all present (camera SDK 4.8.2.2). **Connection test with the camera plugged in** (`tests/check_real_camera.py`; logs in `tests/real_camera_check/`): the connector opened the camera through the SDK and read back the model (MV-CS028-10UM, USB3 Vision), the sensor (1936 × 1464), the pixel format (Mono12) and the ranges (exposure 9 μs–10 s, gain 0–24 dB). Two settings of the camera itself were wrong for measurements, and the test found both: the camera digitised with 8

bits (ADCBitDepth_8), so the Mono12 values came in steps of 17 DN; and the default BlackLevel put the dark level at 0 DN, which cut off the negative half of the dark noise (half of the dark pixels read exactly 0). The connector now selects the deepest ADC mode (ADCBitDepth_12) and sets BlackLevel so that the dark level sits about 60 DN above zero. With these settings, looking at a white wall at 0 dB: mean 819.7 DN at 2 ms and 1575.0 DN at 4 ms, i.e. the signal above the dark level doubles (ratio 2.00); 42 frames/s with the software trigger at 4 ms; a 256×256 ROI and the return to the full sensor work; with the camera covered, the dark level is 62.4 DN and the temporal noise 0.95 DN rms.

(b) The MCP server

mcp_server/camera_server.py is an MCP server (Python MCP SDK, FastMCP, stdio transport). It opens the camera through the connector (CAMERA_KIND=hikrobot for the lab camera, sim or sim-room for the simulated one) and carries out every tool call with it. Each tool is one action of the camera or one reading it returns; the list below is the server's own list_tools:

tool	arguments	what it does
camera_info	–	Describe the camera: vendor, model, sensor size, bit depth (max DN), exposure and gain ranges, the lab limits, and the current settings (exposure, gain, ROI).
set_exposure	exposure_us: number	Set the exposure time in microseconds.
set_gain	gain_db: number	Set the analog gain in dB.
set_roi	x: integer, y: integer, width: integer, height: integer	Set the region of interest: offset (x, y) and size in sensor pixels.
reset_roi	–	Use the full sensor again.
set_illumination	level: number	Light source level 0..1 on the simulated camera only (0 = dark, for dark frames).
set_black_level	value: integer	Set the black level: the offset added before digitisation, in the camera's own units (on the Hikrobot camera a LARGER value gives a LOWER dark level; on the simulated camera it is the dark level in DN).
set_adc_bit_depth	bits: integer	Set the bits of the analogue-to-digital converter (8 or 12).
read_temperature	–	The camera's device temperature in degrees C (dark current roughly doubles every 6-8 C, so long series and dark frames should report it).
save_tiff	frame_file: string	Save a captured frame stack (.npy) as a 16-bit TIFF, one page per frame, for ImageJ/Fiji or other software.
capture	n_frames: integer (opt.), label: string (opt.), regions: array (opt.)	Acquire n_frames frames with the current settings and save them.
capture_series	n_frames: integer, interval_s: number, label: string (opt.), regions: array (opt.)	Stability measurement: n_frames frames, one every interval_s seconds, with the current settings.
analyze_frame	frame_file: string, black_level_dn: number (opt.), region: object (opt.)	Speckle and beam measures of a saved frame (first frame of the file): speckle contrast and grain size (FWHM of the intensity autocorrelation) inside 'region' (or the whole frame), and the beam centroid and D4sigma widths.

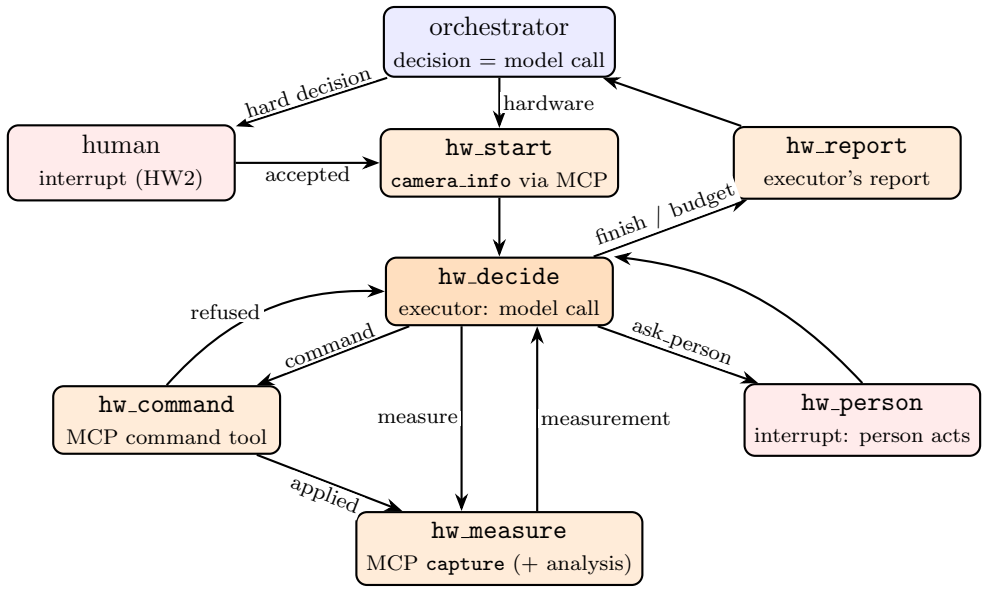
Every tool returns ok and its result, or ok=false and the reason (out of the camera's range, above a lab limit, device error), so a refused command reaches the agent instead of crashing the run. The lab limits (maximum exposure, gain, frames, series length) are set when the server starts. Every call is appended to calls.jsonl (time, tool, arguments, result): this file is the record of what the instrument did, and the tables and sequence diagrams in (f) are generated from it. Frames are saved as .npy with a preview .png.

(c) One agent holds the tools

The roles are those of Homeworks 1 and 2: orchestrator, scientist, general agent, executor, plus the human stop. Only the **executor** receives the MCP server: `agent/mcp_bridge.py` starts the server, keeps one MCP session open for the whole run (so the camera keeps its settings between calls) and gives the executor the server's tools with the schemas the server publishes. No other role can reach the camera. The decision to call a tool is a model call: at each step of a hardware turn the executor's model reads the dialogue (the request, the scientist's plan, every earlier command and measurement) and returns a structured decision `HWDecision(action, command, command_args, n_frames, label, regions, analyze, black_level_dn, analysis_region, question, summary, reason)`, where `action` is *command* (one command tool, then a capture), *measure*, *ask_person* or *finish*. The graph then makes exactly that MCP call. The executor also keeps its software tools from the earlier homeworks: the coding tool (scripts that analyse the saved frames) and the focusing tool of the Homework 2 simulated bench, whose camera model (read noise, full well, signal level, light-source noise) can be set per experiment from the real camera's measured numbers.

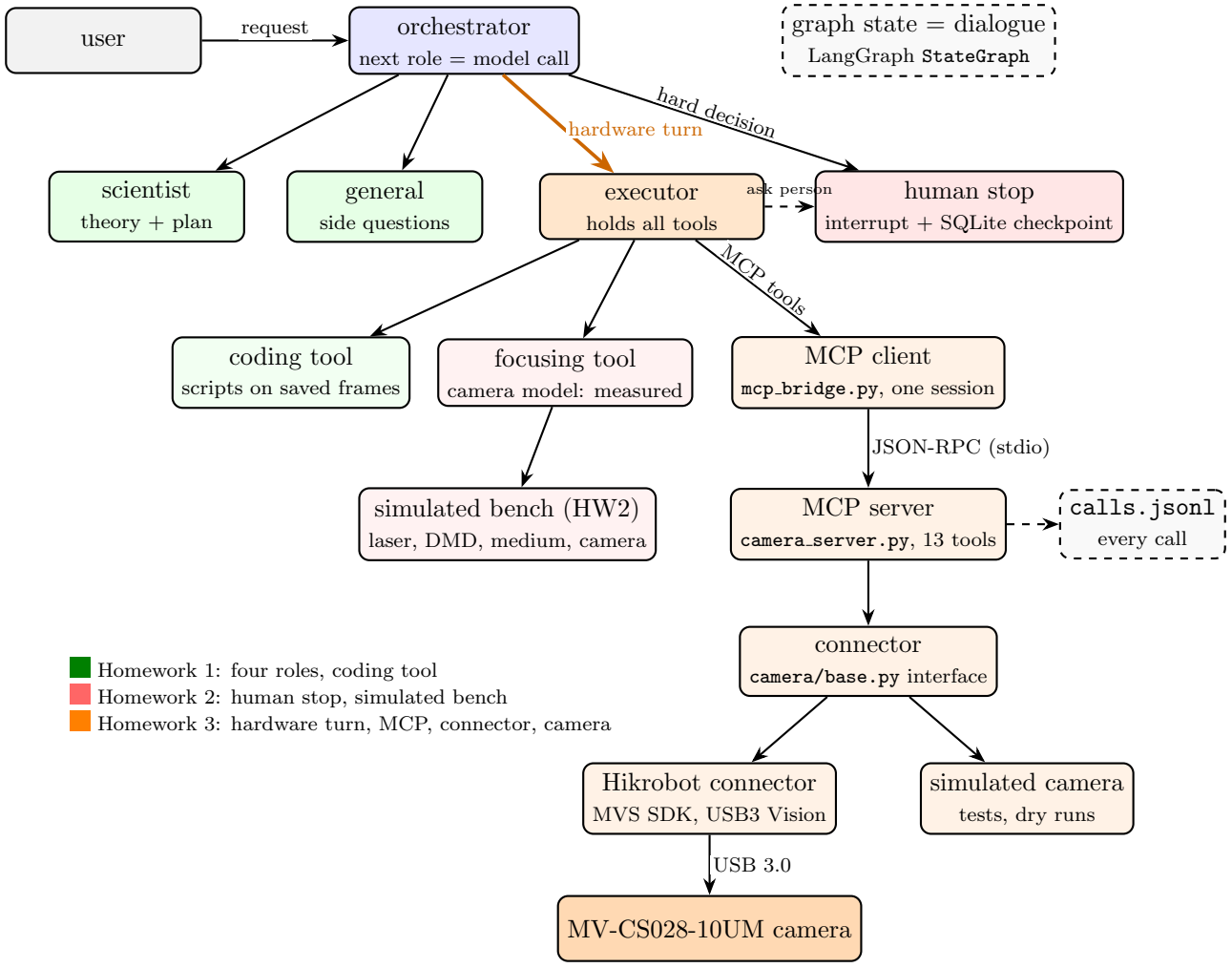
(d) Hardware in the loop

The hardware turn is built like the human stop of Homework 2: the orchestrator's decision (a model call) can choose a new destination, `hardware`, when the next step needs the instrument. The graph then gives the turn to the executor in a loop of nodes:



Every node writes one turn into the dialogue: the instrument's reply to a command (the value it applied, or why it refused) and every measurement (frame statistics, temporal noise, region values, speckle/beam measures, file name). The next `hw_decide` reads those turns before it chooses again: *decide → command the hardware → receive the measurement → decide again.* A refused command goes straight back to `hw_decide`. The turn ends when the executor chooses *finish* (or after 12 steps); `hw_report` then writes the settings and measurements the camera returned, marked VERIFIED, and the turn returns to the orchestrator, which may finish only after such a record exists. The human stop stays: a hardware turn that changes settings or runs a long acquisition is a hard decision for which the orchestrator may ask the person first (accepted → the turn runs; refused → it does not, and a record says so), and inside the turn *ask_person* pauses the graph for what only a person can do, e.g. block the light for dark frames on the lab bench, where the light is not under software control. Both stops are LangGraph interrupts on a checkpointed thread.

(e) Architecture of the system so far



The scientist, the general agent and the human answer the orchestrator, and every specialist turn returns to it; all turns are written into the one dialogue that is the graph's state. Sequence diagrams of the runs are in (f), one per run.

(f) Use cases and the demo

Setup at the lab. The MV-CS028-10UM on the laptop's USB 3 port, without a lens (a sheet of paper over the sensor opening diffuses the light), looking at a white wall next to a window under LED room lights; no laser. Every run is `python run_hw3.py --camera hikrobot --thread <name> --request "..."` (Claude Sonnet 5.5 through the Claude Code CLI); the lab limits were 500 ms exposure, 24 dB gain and 16 frames per capture. When the agent asked for something only a person can do (block or unblock the light) or, with `--human`, for approval of a hard decision, the person at the bench answered and the reply was passed to the run as given (in English). The tables, sequence diagrams and frames below are generated from each run's `calls.jsonl` and transcript by `agent/render_hw3.py`.

request	decisions	hw steps	MCP calls	verified result
Set the camera so that the white surface uses most of the camera's range without saturating, and report the exposure and gain you chose.	2	4	7	9400 us, 0 dB: max 3592 DN, 0 saturated px

request	decisions	hw steps	MCP calls	verified result
The camera looks at a white surface under the lab's room lights, which may flicker. Find an exposure at which the frame-to-frame brightness is stable, with the image well exposed but not saturated, and report how strong the flicker is at a short exposure and at the exposure you chose.	3	13	23	10000 us, 0 dB: max 2497 DN, 0 saturated px
At an exposure free of the light flicker, measure how stable the brightness is over two minutes and report the drift.	4	8	11	script drift_analysis.py: SLOPE_PCT_PER_MIN=-3.43, TOTAL_CHANGE_PCT=-6.859, FLICKER_NFRAMES_5ms=16, FLICKER_MEAN_DN_5ms=1157
Measure this camera's read noise, conversion gain and full well capacity in electrons. Then use these measured numbers as the camera of the Homework 2 focusing bench and predict the focusing enhancement we can expect with this camera, choosing the bench signal level so that the focus does not saturate it.	5	12	20	focusing tool (simulated bench), experiment 4: enhancement 11
Set the gain to 12 dB and choose the exposure so that the wall is at about half of the camera's range, then report the exposure and the temporal noise.	3	10	11	3250 us, 11.991 dB: max 2163 DN, 0 saturated px
Set the gain to 12 dB and choose the exposure so that the wall is at about half of the camera's range, then report the exposure and the temporal noise.	3	3	6	4090 us, 11.991 dB: max 2201 DN, 0 saturated px
Do not use the camera. The lab camera (Hikrobot MV-CS028-10UM) was measured in an earlier run: conversion gain 6.14 e-/DN, read noise 5.78 e-, full well at least 24766 e-. With the focusing tool, compare the focusing enhancement at the Homework 2 settings (macropixel 4, focus_size 3, bg_inner 6, bg_outer 20, bg_regions 8, iterations 40000, flips 4, remeasure true) for the bench's default camera and for this measured camera, with the bench signal level chosen so that the focus does not saturate this camera. Report both enhancements.	3	0	0	focusing tool (simulated bench), experiment 4: enhancement 30.63

What the runs found:

- **Auto-exposure:** from a first frame at 4 ms (peak 1587 DN) the executor scaled the exposure to 9 ms and then 9.4 ms: peak 3592 DN (87.7 % of full scale), no saturated pixel, gain left at 0 dB.
- **Flicker:** the LED lights hardly flicker. The frame-to-frame spread of the mean was 0.14 % at 1 ms and 0.05 % at 10 ms; the agent chose 10 ms (one 100 Hz period, 59 % of full scale) and stated that the flicker is weak.
- **Stability:** 121 frames over 120 s at 10 ms: the brightness fell smoothly by 3.43 %/min (6.9 % in total, residual scatter 0.14 %). The agent asked the person to block the light for the dark level (64.1 DN) and did not attribute the drift to the camera; with daylight from the window in the evening, the light itself was changing.

- **Noise model and focusing prediction:** the person blocked and then unblocked the light when asked. From dark frames and a photon-transfer series the agent measured a conversion gain of 6.14 e^-/DN, a read noise of 5.78 e^- and a full well of at least 24 766 e^- (limited by the 12-bit ADC, so a lower bound). With these numbers as the bench's camera and a signal level chosen so that the focus stays at about 66 % of the full well, the focusing tool predicted an enhancement of 10.8 (8.4 against the far field) for the quick configuration the agent chose (64 segments, 6000 iterations). The agent itself noted that this value is set by the bench configuration rather than by the camera; for binary-amplitude shaping with 64 segments the expected enhancement is about $N/2\pi \approx 10$. A follow-up run without the camera (use case 7) therefore compared both cameras at the Homework 2 settings (256 segments, 40 000 iterations, the same signal of 400 e^-/px): the bench's default camera gave 33.55 and the measured lab camera 30.63 (at 100 e^-/px: 26.45 and 25.12), with no saturated pixel. These values are against the background ring of 6–20 px, which the search can darken; against the far field (focus over the mean speckle outside the ring, the standard definition) the same runs gave 21.7 and 26.5 (at 100 e^-/px: 21.1 and 21.4), the opposite order. With one run per setting, the difference between the two cameras is therefore within the run-to-run spread: at these signal levels the measured lab camera does not limit the focusing.
- **Human stops, first attempt:** the person approved the hardware turn. Inside it the agent asked for the light to be blocked for dark frames; the person replied without blocking it (having not been told what the dark frames were for). The agent did not trust the reply: it checked each "dark" set, saw that it was as bright as the wall (2054 vs. 2076 DN), asked again (three requests in all), and finally reported the dark level and read noise as undetermined instead of using bright frames; the person approved its decision to finish. **Second run** (recorded): the orchestrator stopped twice for approval (before the hardware turn and before finishing); the person answered *yes* both times; result 4.09 ms at 12 dB, wall at 49.6 % of full scale, temporal noise 35.9 DN rms.

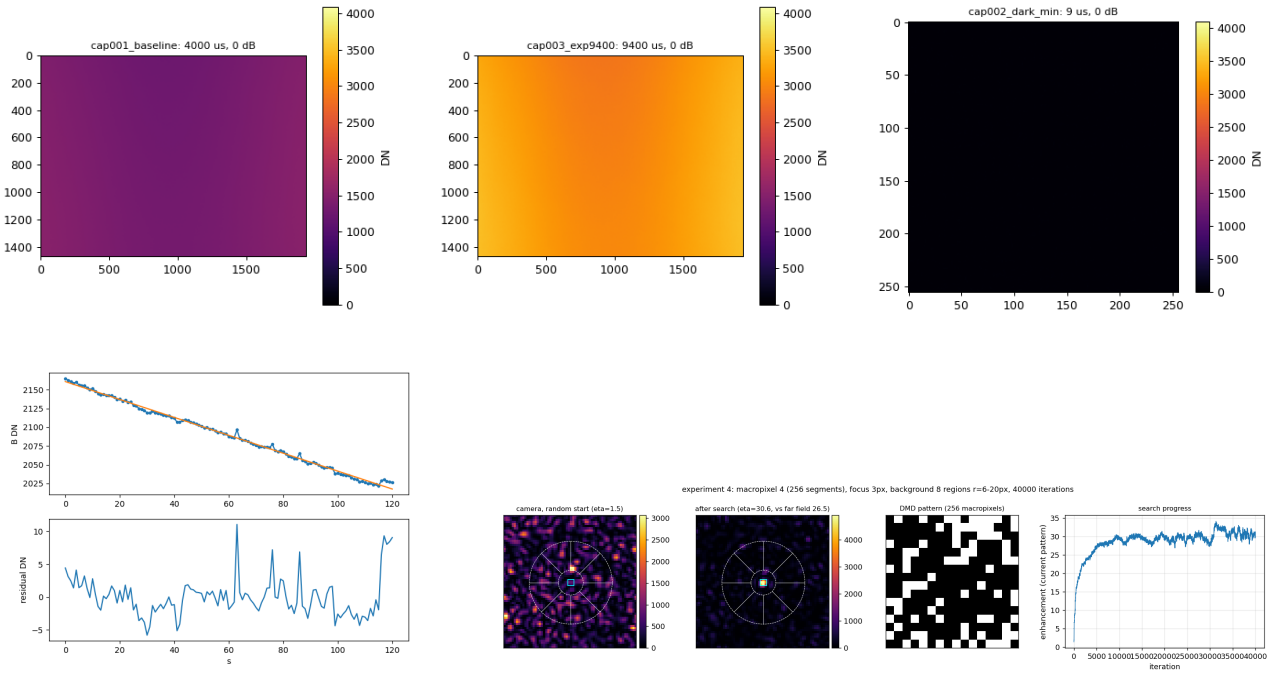


Figure 1: What the instrument returned (previews saved by the MCP server, 0–4095 DN). Top: use case 1, the first frame at 4 ms and the frame at the 9.4 ms the agent chose (the white wall seen through the paper diffuser, brighter towards the sides); use case 4, a dark frame taken after the person blocked the light. Bottom left: use case 3, the frame-mean brightness over 120 s and the residuals of the linear fit (the agent's analysis script). Bottom right: use case 7, the Homework 2 bench with the measured camera model: speckle at the random start, the focus after 40 000 iterations, the DMD pattern and the search progress (simulation).

Demo videos (screen recordings, cropped to the live camera view and the run's terminal, no sound), in the repository folder `video/` and attached to the email: `hw3_demo_real_camera.mp4` (59 s, use case 1: the executor decides, calls `set_exposure` and `capture` through MCP, and the camera's frame and statistics come back) and `hw3_demo_human_stop.mp4` (91 s, use case 6: the graph pauses twice and the person approves).

Three use cases are shown in full below (tool-call table and sequence diagram); the other four follow in compact form (request, every tool call with what the instrument did, verified result, a smaller sequence diagram). All seven are in the repository with their transcripts and `calls.jsonl`.

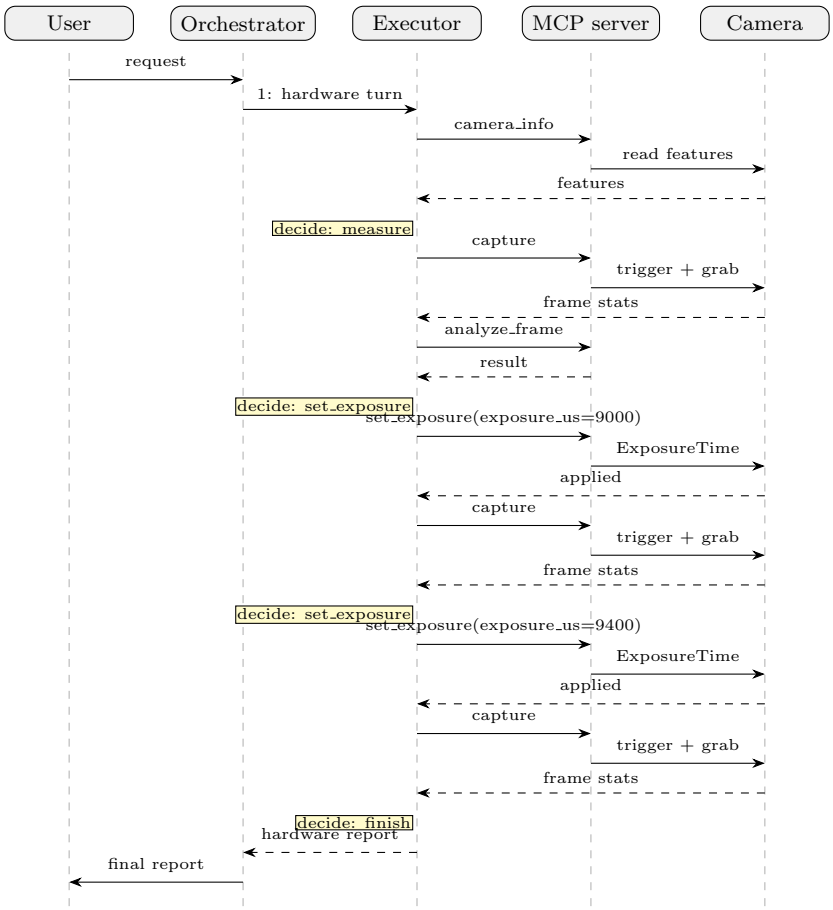
Use case 1 (real camera): auto-exposure (first video)

Request: *Set the camera so that the white surface uses most of the camera's range without saturating, and report the exposure and gain you chose.*
Camera: Hikrobot MV-CS028-10UM (real hardware); 2 orchestrator decisions, 4 hardware decisions, 7 MCP tool calls.

Tool calls and what the instrument did.

#	tool	arguments	what the instrument did
1	`camera_info`	-	MV-CS028-10UM, 1936x1464, 12-bit; exposure 4000.0 us, gain 0.0 dB
2	`capture`	n_frames=2, label=baseline	2 frame(s) at 4000 us, 0 dB: mean 1380.61, max 1587 DN, 0 saturated px; temporal noise 14.7868 DN
3	`analyze_frame`	frame_file=cap001_baseline.npy, black_level_dn=0.0	speckle contrast 0.0528, grain 645.079 x 14425.666 px; beam centroid (970.36, 738.09) px
4	`set_exposure`	exposure_us=9000.0	exposure set to 9000 us
5	`capture`	n_frames=2, label=exp9000	2 frame(s) at 9000 us, 0 dB: mean 3022.47, max 3455 DN, 0 saturated px; temporal noise 22.0274 DN
6	`set_exposure`	exposure_us=9400.0	exposure set to 9400 us
7	`capture`	n_frames=2, label=exp9400	2 frame(s) at 9400 us, 0 dB: mean 3152.33, max 3592 DN, 0 saturated px; temporal noise 22.4913 DN

Sequence diagram. Solid arrows: messages and commands; dashed: replies and measurements; boxes: the executor's decisions (model calls). Numbers are the orchestrator's decision index.



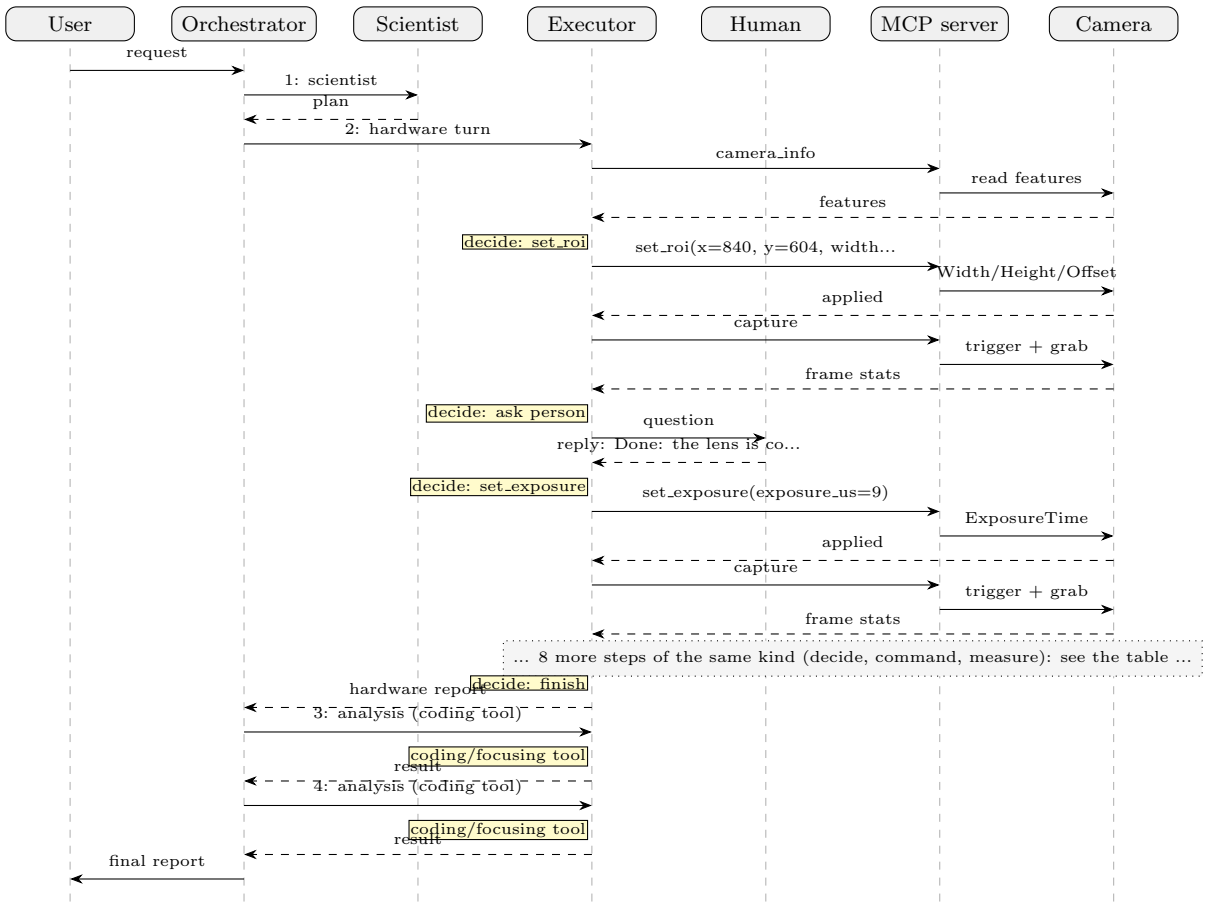
Use case 4 (real camera): noise model and focusing prediction, the person blocks the light

Request: *Measure this camera's read noise, conversion gain and full well capacity in electrons. Then use these measured numbers as the camera of the Homework 2 focusing bench and predict the focusing enhancement we can expect with this camera, choosing the bench signal level so that the focus does not saturate it.*
Camera: Hikrobot MV-CS028-10UM (real hardware); 5 orchestrator decisions, 12 hardware decisions, 20 MCP tool calls.

Tool calls and what the instrument did.

#	tool	arguments	what the instrument did
1	`camera_info`	-	MV-CS028-10UM, 1936x1464, 12-bit; exposure 10000.0 us, gain 0.0 dB
2	`set_roi`	x=840, y=604, width=256, height=256	ROI 256x256 at (840, 604)
3	`capture`	n_frames=1, label=roi_set	1 frame(s) at 10000 us, 0 dB: mean 63.46, max 74 DN, 0 saturated px
4	`server_restart_restore`	-	server restarted; settings restored {'roi': [840, 604, 256, 256]}
5	`set_exposure`	exposure_us=9.0	exposure set to 9 us
6	`capture`	n_frames=16, label=dark_min	16 frame(s) at 9 us, 0 dB: mean 62.54, max 70 DN, 0 saturated px; temporal noise 0.9414 DN
7	`set_exposure`	exposure_us=100000.0	exposure set to 100000 us
8	`capture`	n_frames=10, label=dark_long	10 frame(s) at 100000 us, 0 dB: mean 63.37, max 153 DN, 0 saturated px; temporal noise 1.0984 DN
9	`server_restart_restore`	-	server restarted; settings restored {'exposure_us': 100000.0, 'roi': [840, 604, 256, 256]}
10	`capture`	n_frames=2, label=flat_test_100ms	2 frame(s) at 100000 us, 0 dB: mean 4093.0, max 4093 DN, 0 saturated px; temporal noise 0.0 DN
11	`set_exposure`	exposure_us=20000.0	exposure set to 20000 us
12	`capture`	n_frames=2, label=flat_test_20ms	2 frame(s) at 20000 us, 0 dB: mean 3550.32, max 3667 DN, 0 saturated px; temporal noise 23.8926 DN
13	`set_exposure`	exposure_us=10000.0	exposure set to 10000 us
14	`capture`	n_frames=2, label=flat_10ms	2 frame(s) at 10000 us, 0 dB: mean 1819.78, max 1900 DN, 0 saturated px; temporal noise 17.0794 DN
15	`set_exposure`	exposure_us=3000.0	exposure set to 3000 us
16	`capture`	n_frames=8, label=flat_3ms	8 frame(s) at 3000 us, 0 dB: mean 587.78, max 632 DN, 0 saturated px; temporal noise 9.4106 DN
17	`set_exposure`	exposure_us=23000.0	exposure set to 23000 us
18	`capture`	n_frames=4, label=flat_23ms	4 frame(s) at 23000 us, 0 dB: mean 4071.6, max 4093 DN, 0 saturated px; temporal noise 19.4636 DN
19	`set_exposure`	exposure_us=6000.0	exposure set to 6000 us
20	`capture`	n_frames=6, label=flat_6ms	6 frame(s) at 6000 us, 0 dB: mean 1107.92, max 1168 DN, 0 saturated px; temporal noise 13.2053 DN

Sequence diagram. Solid arrows: messages and commands; dashed: replies and measurements; boxes: the executor's decisions (model calls). Numbers are the orchestrator's decision index.


User
Orchestrator
Scientist
Executor
Human
MCP server
Camera
request
1: scientist
plan
2: hardware turn
camera_info
read features
features
decide: set_roi
set_roi(x=840, y=604, width...
Width/Height/Offset
applied
capture
trigger + grab
frame stats
decide: ask person
question
reply: Done: the lens is co...
decide: set_exposure
set_exposure(exposure_us=9)
ExposureTime
applied
capture
trigger + grab
frame stats
... 8 more steps of the same kind (decide, command, measure): see the table ...
decide: finish
hardware report
3: analysis (coding tool)
coding/focusing tool
result
4: analysis (coding tool)
coding/focusing tool
result
final report

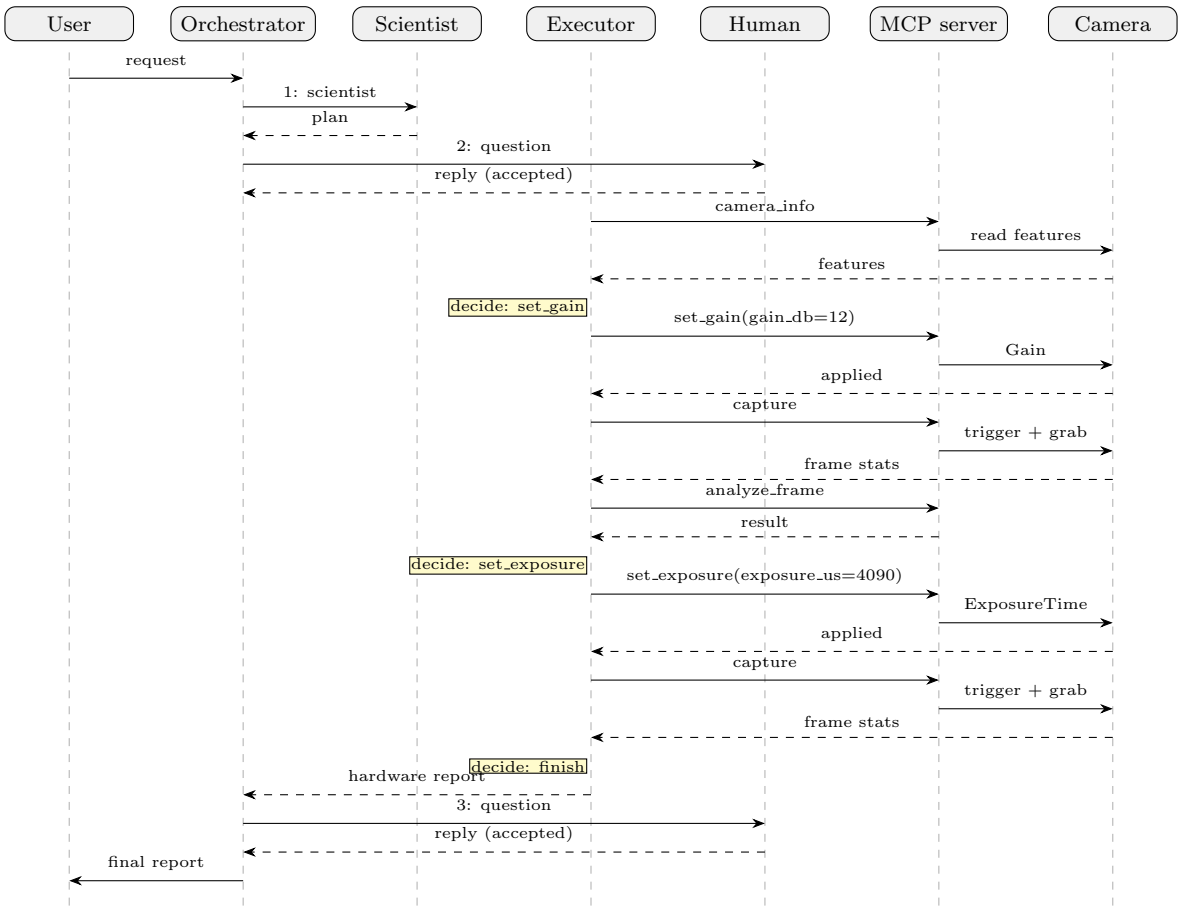
Use case 6 (real camera): human stops (second video)

Request: *Set the gain to 12 dB and choose the exposure so that the wall is at about half of the camera's range, then report the exposure and the temporal noise.*
Camera: Hikrobot MV-CS028-10UM (real hardware); 3 orchestrator decisions, 3 hardware decisions, 6 MCP tool calls.

Tool calls and what the instrument did.

#	tool	arguments	what the instrument did
1	`camera_info`	-	MV-CS028-10UM, 1936x1464, 12-bit; exposure 3250.0 us, gain 11.991 dB
2	`set_gain`	gain_db=12.0	gain set to 11.991 dB
3	`capture`	n_frames=5, label=iter_1	5 frame(s) at 3250 us, 11.991 dB: mean 1626.1, max 1765 DN, 0 saturated px; temporal noise 32.2099 DN
4	`analyze_frame`	frame_file=cap001_iter_1.npy, black_level_dn=0.0	speckle contrast 0.0204, grain 0.999 x 1.034 px; beam centroid (127.5, 127.6) px
5	`set_exposure`	exposure_us=4090.0	exposure set to 4090 us
6	`capture`	n_frames=5, label=iter_2	5 frame(s) at 4090 us, 11.991 dB: mean 2030.38, max 2201 DN, 0 saturated px; temporal noise 35.9004 DN

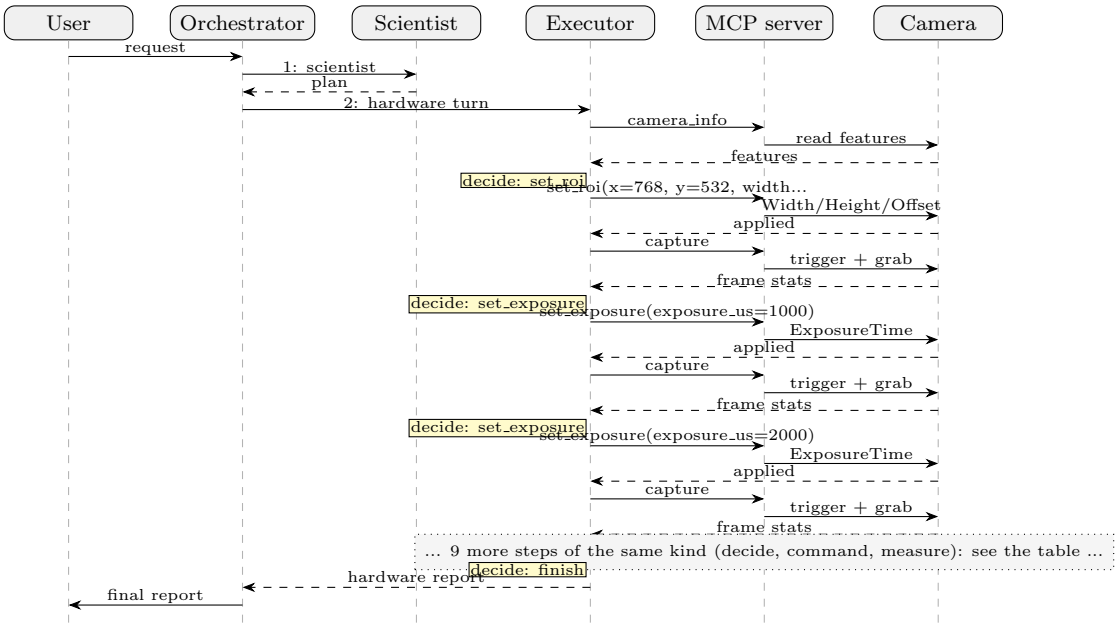
Sequence diagram. Solid arrows: messages and commands; dashed: replies and measurements; boxes: the executor's decisions (model calls). Numbers are the orchestrator's decision index.



Use case 2 (real camera): flicker-free exposure

Request: *The camera looks at a white surface under the lab's room lights, which may flicker. Find an exposure at which the frame-to-frame brightness is stable, with the image well exposed but not saturated, and report how strong the flicker is at a short exposure and at the exposure you chose.*
Camera: Hikrobot MV-CS028-10UM (real hardware); 3 orchestrator decisions, 13 hardware decisions, 23 MCP tool calls.
Tool calls and what the instrument did: (1) `camera_info` → MV-CS028-10UM, 1936x1464, 12-bit; exposure 9400.0 us, gain 0.0 dB; (2) `set_roi` x=768, y=532, width=400, height=400 → ROI 400x400 at (768, 532); (3) `capture` n_frames=1 → 1 frame(s) at 9400 us, 0 dB: mean 2313.58, max 2409 DN, 0 saturated px; (4) `set_exposure` exposure_us=1000.0 → exposure set to 1000 us; (5) `capture` n_frames=16, label=short_1000 → 16 frame(s) at 1000 us, 0 dB: mean 301.09, max 331 DN, 0 saturated ...; (6) `set_exposure` exposure_us=2000.0 → exposure set to 2000 us; (7) `capture` n_frames=16, label=short_2000 → 16 frame(s) at 2000 us, 0 dB: mean 540.24, max 583 DN, 0 saturated ...; (8) `set_exposure` exposure_us=5000.0 → exposure set to 5000 us; (9) `capture` n_frames=16, label=scan_5000 → 16 frame(s) at 5000 us, 0 dB: mean 1251.61, max 1317 DN, 0 saturate...; (10) `set_exposure` exposure_us=8333.0 → exposure set to 8333 us; (11) `capture` n_frames=16, label=scan_8333 → 16 frame(s) at 8333 us, 0 dB: mean 2041.56, max 2130 DN, 0 saturate...; (12) `set_exposure` exposure_us=10000.0 → exposure set to 10000 us; (13) `capture` n_frames=16, label=scan_10000 → 16 frame(s) at 10000 us, 0 dB: mean 2436.85, max 2534 DN, 0 saturat...; (14) `set_exposure` exposure_us=12000.0 → exposure set to 12000 us; (15) `capture` n_frames=16, label=scan_12000 → 16 frame(s) at 12000 us, 0 dB: mean 2902.14, max 3015 DN, 0 saturat...; (16) `set_exposure` exposure_us=16667.0 → exposure set to 16667 us; (17) `capture` n_frames=16, label=scan_16667 → 16 frame(s) at 16667 us, 0 dB: mean 3994.7, max 4093 DN, 0 saturate...; (18) `set_exposure` exposure_us=1000.0 → exposure set to 1000 us; (19) `capture` n_frames=1, label=short_1000_series → 1 frame(s) at 1000 us, 0 dB: mean 297.33, max 330 DN, 0 saturated px; (20) `capture_series` n_frames=16, interval_s=0.137, label=... → 16 frames over 2.06 s: mean 296.892 DN, rel. std 0.1411%, drift -0....; (21) `set_exposure` exposure_us=10000.0 → exposure set to 10000 us; (22) `capture` n_frames=1, label=series_10000_pre → 1 frame(s) at 10000 us, 0 dB: mean 2398.35, max 2497 DN, 0 saturate...; (23) `capture_series` n_frames=16, interval_s=0.137, label=... → 16 frames over 2.069 s: mean 2390.202 DN, rel. std 0.0494%, drift 0....
Verified result: 10000 us, 0 dB: max 2497 DN, 0 saturated px.

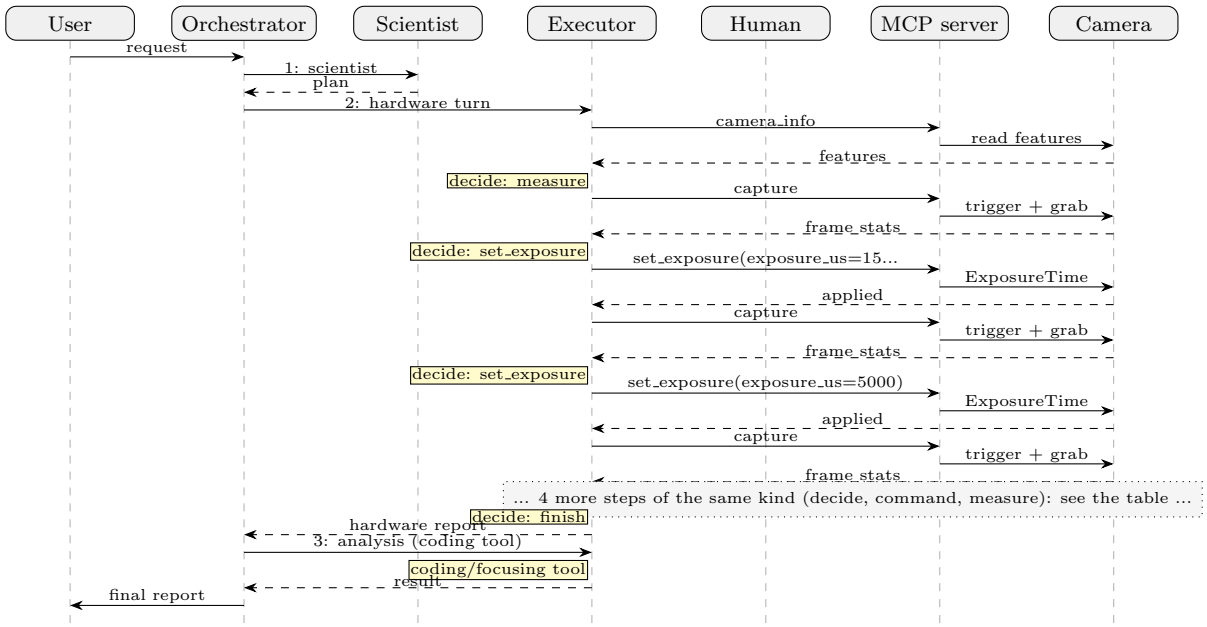
Sequence diagram (as above; long hardware turns shortened).



Use case 3 (real camera): stability, the person blocks the light

Request: *At an exposure free of the light flicker, measure how stable the brightness is over two minutes and report the drift.*
Camera: Hikrobot MV-CS028-10UM (real hardware); 4 orchestrator decisions, 8 hardware decisions, 11 MCP tool calls.
Tool calls and what the instrument did: (1) `camera_info` → MV-CS028-10UM, 1936x1464, 12-bit; exposure 10000.0 us, gain 0.0 dB; (2) `capture` n_frames=4, label=baseline_10ms → 4 frame(s) at 10000 us, 0 dB: mean 2274.07, max 2368 DN, 0 saturate...; (3) `set_exposure` exposure_us=15000.0 → exposure set to 15000 us; (4) `capture` n_frames=16, label=scan_15ms → 16 frame(s) at 15000 us, 0 dB: mean 3362.9, max 3497 DN, 0 saturate...; (5) `set_exposure` exposure_us=5000.0 → exposure set to 5000 us; (6) `capture` n_frames=16, label=scan_5ms → 16 frame(s) at 5000 us, 0 dB: mean 1156.75, max 1221 DN, 0 saturate...; (7) `set_exposure` exposure_us=10000.0 → exposure set to 10000 us; (8) `capture` n_frames=16, label=scan_10ms → 16 frame(s) at 10000 us, 0 dB: mean 2244.14, max 2359 DN, 0 saturat...; (9) `capture_series` n_frames=121, interval_s=1.0, label=d... → 121 frames over 120.014 s: mean 2153.35 DN, rel. std 1.942%, drift ...; (10) `server_restart_restore` → server restarted; settings restored {'exposure_us': 10000.0}; (11) `capture` n_frames=16, label=dark_after → 16 frame(s) at 10000 us, 0 dB: mean 63.4, max 75 DN, 0 saturated px....
Verified result: script drift_analysis.py: SLOPE_PCT_PER_MIN=-3.43, TOTAL_CHANGE_PCT=-6.859, FLICKER_NFRAMES_5ms=16, FLICKER_MEAN_DN_5ms=1157.

Sequence diagram (as above; long hardware turns shortened).

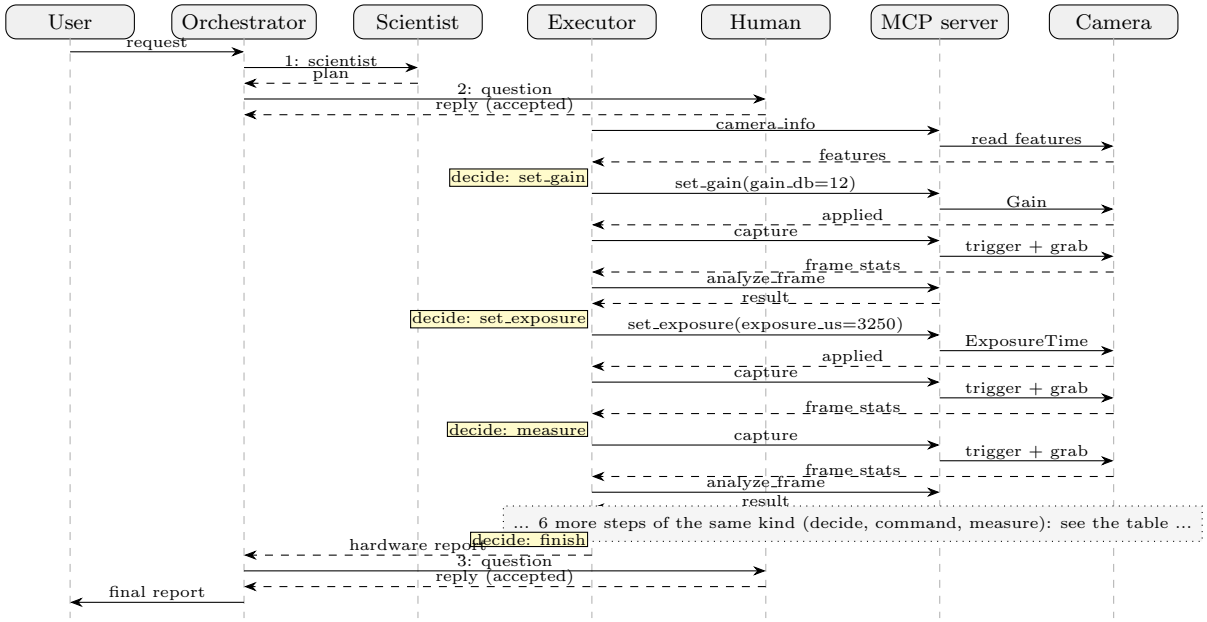


Use case 5 (real camera): human stops, first attempt

Request: *Set the gain to 12 dB and choose the exposure so that the wall is at about half of the camera's range, then report the exposure and the temporal noise.*
Camera: Hikrobot MV-CS028-10UM (real hardware); 3 orchestrator decisions, 10 hardware decisions, 11 MCP tool calls.
Tool calls and what the instrument did: (1) `camera_info` → MV-CS028-10UM, 1936x1464, 12-bit; exposure 6000.0 us, gain 0.0 dB; (2) `set_gain` gain_db=12.0 → gain set to 11.991 dB; (3) `capture` n_frames=2, label=gain12_trial → 2 frame(s) at 6000 us, 11.991 dB: mean 3776.93, max 4009 DN, 0 satu...; (4) `analyze_frame` frame_file=cap001_gain12_trial.npy, b... → speckle contrast 0.014, grain 1.018 x 1.055 px; beam centroid (127....; (5) `set_exposure` exposure_us=3250.0 → exposure set to 3250 us; (6) `capture` n_frames=2, label=gain12_exp3250 → 2 frame(s) at 3250 us, 11.991 dB: mean 2076.49, max 2238 DN, 0 satu...; (7) `capture` n_frames=16, label=wall_gain12_exp3250 → 16 frame(s) at 3250 us, 11.991 dB: mean 2075.86, max 2241 DN, 0 sat...; (8) `analyze_frame` frame_file=cap003_wall_gain12_exp3250... → speckle contrast 0.0183, grain 1.016 x 1.036 px;

beam centroid (127...; (9) `capture` n_frames=16, label=dark_gain12_exp3250 → 16 frame(s) at 3250 us, 11.991 dB: mean 2054.24, max 2207 DN, 0 sat...; (10) `capture` n_frames=16, label=dark_gain12_exp325... → 16 frame(s) at 3250 us, 11.991 dB: mean 2024.53, max 2192 DN, 0 sat...; (11) `capture` n_frames=16, label=dark_gain12_exp325... → 16 frame(s) at 3250 us, 11.991 dB: mean 1996.24, max 2163 DN, 0 sat....
Verified result: 3250 us, 11.991 dB: max 2163 DN, 0 saturated px.

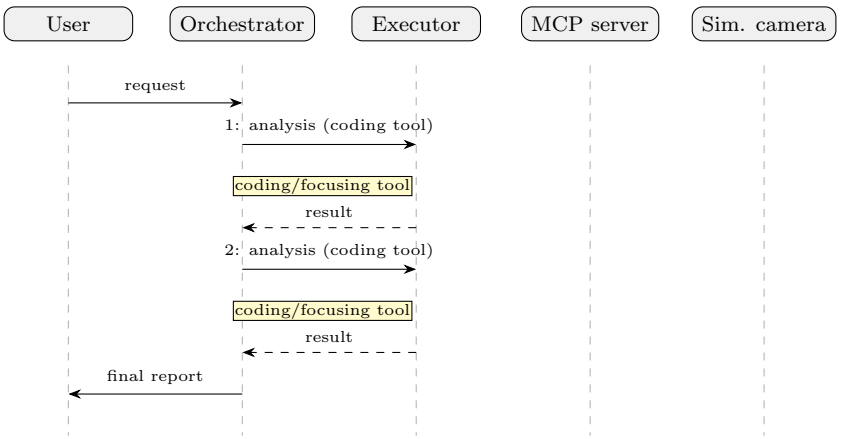
Sequence diagram (as above; long hardware turns shortened).



Use case 7 (no camera): focusing prediction at the Homework 2 settings, with the measured camera

Request: *Do not use the camera. The lab camera (Hikrobot MV-CS028-10UM) was measured in an earlier run: conversion gain 6.14 e-/DN, read noise 5.78 e-, full well at least 24766 e-. With the focusing tool, compare the focusing enhancement at the Homework 2 settings (macropixel 4, focus_size 3, bg_inner 6, bg_outer 20, bg_regions 8, iterations 40000, flips 4, remeasure true) for the bench's default camera and for this measured camera, with the bench signal level chosen so that the focus does not saturate this camera. Report both enhancements.*
Camera: not used in this run (simulated bench only); 3 orchestrator decisions, 0 hardware decisions, 0 MCP tool calls.
Focusing tool calls: experiment 1: {"macropixel": 4, "focus_size": 3, "bg_inner": 6, "bg_outer": 20, "bg_regions": 8, "iterations": 40000, "flips": 4, "flips_...; experiment 2: {"macropixel": 4, "focus_size": 3, "bg_inner": 6, "bg_outer": 20, "bg_regions": 8, "iterations": 40000, "flips": 4, "flips_...; experiment 3: {"macropixel": 4, "focus_size": 3, "bg_inner": 6, "bg_outer": 20, "bg_regions": 8, "iterations": 40000, "flips": 4, "flips_...; experiment 4: {"macropixel": 4, "focus_size": 3, "bg_inner": 6, "bg_outer": 20, "bg_regions": 8, "iterations": 40000, "flips": 4, "flips_....
Verified result: focusing tool (simulated bench), experiment 4: enhancement 30.63.

Sequence diagram (as above; long hardware turns shortened).



Dry runs on the simulated camera

Before the lab session the same system ran six requests on the simulated camera behind the same MCP server (`--camera sim` or `sim-room`; runs `agent/runs/uc1`–`uc6`). Because the simulated camera's true parameters are known, these runs check the method: the agent's photon-transfer measurements gave 2.30 and 2.41 e^-/DN and 5.66 and 6.04 e^- read noise (true: 2.40 e^-/DN and 6.0 e^-), and in the room-light scene it found the 100 Hz flicker (22 % at 1 ms, 0.27 % at 30 ms).

(g) Added after the deadline extension

The lab session showed that two camera settings decide whether a noise measurement is right: with the camera's default 8-bit ADC a Mono12 frame has only about 240 levels (steps of 16 DN, larger than the read noise), and with the default black level the dark level sits at 0 DN and half of the dark noise is clipped. The connector fixed both at start-up. They are now **tools the agent can use**, together with two more, so the MCP server has 13 tools (table in (b)):

tool	what it does	lab camera (GenICam node)	simulated camera
`set_black_level`	offset added before digitisation; returns the applied value	`BlackLevel` (integer; a larger value gives a lower dark level: 240 → 0 DN, 180 → 62 DN)	the dark level in DN
`set_adc_bit_depth`	8 or 12 bits of the ADC	`ADCBitDepth` (`Bits_8`/`Bits_12`), changed with the acquisition stopped	rounds to 256 levels of the 12-bit scale
`read_temperature`	device temperature in °C (dark current depends on it)	`DeviceTemperature`	warms from 38 to 41 °C in about 10 minutes
`save_tiff`	a capture as a 16-bit multi-page TIFF for ImageJ/Fiji	on the server (Pillow)	the same

The new settings are part of the camera interface (`camera/base.py`), so every connector offers them or refuses them with a reason. A restarted server restores them from `calls.jsonl` like exposure and gain, every capture reports them in its settings, and in the hardware turn `read_temperature` and `save_tiff` are commands after which no frame is captured; the VERIFIED record of a hardware turn lists the temperatures read and the TIFF files.

Packaging. `hw3.py` is one entry point: `test` (all offline tests with a summary), `check-camera` (the MVS SDK, then the plugged-in camera), `tools` (the server's tools as an MCP client sees them), `run`, `live`, `demo`, and `mcp-config` (the entry that adds the camera server to any MCP client, e.g. Claude Desktop). Every model call through the Claude Code CLI is now counted (calls, input and output tokens, the CLI's cost estimate), and `model@low` lowers the model's thinking effort (36 % fewer output tokens in a test). The MCP client always uses the Windows proactor event loop, so the camera server also starts inside other programs, such as the web app of Homeworks 4/5.

Tests (`python hw3.py test`, simulated camera; all three suites pass). With the 8-bit ADC every value of a dark frame is a multiple of 4095/255 (16, 32, 48, 64 DN, ...); a black level of 100 gives a dark level of 100.0 DN; the TIFF has one page per frame, equal to the frames; the device temperature reads 38.07 °C; and a restarted server restores the 8-bit ADC, the black level of 100 and the gain of 6 dB.

Used by the agent. In a run of the lab assistant of Homeworks 4/5 on the simulated camera (request: measure the noise model and predict the focusing enhancement with it), the instrument team itself set the ADC to 12 bits and the black level to 40 before its photon-transfer measurement, switched the light off with `set_illumination` for 50 dark frames, and reported a conversion gain of 2.401 e^-/DN (true value 2.40), a dark noise of 2.52 DN and a full well of at least 9737 e^- (true 10 000 e^-, not reached up to 30 ms). It marked its read noise of 6.04 e^- (true 6.0 e^-) as not verified because a cross-check failed; the optics team then predicted an enhancement of 5.3 to 5.7 against the far field (5.4 to 6.4 against the background ring) at 64 segments and 1000 iterations with this camera model.

On the lab camera the new connector functions use the same SDK calls as the connector's start-up, which ran at the lab (`SetIntValueEx` for `BlackLevel`, `SetEnumValue` for `ADCBitDepth`), plus `GetFloatValue` for `DeviceTemperature`; `tests/check_real_camera.py` now also checks them (black level up and back, 8- and 12-bit frames, temperature). This check runs at the next lab session (18 October), before the live demonstration of Homeworks 4/5.